■ kubectl & helm

Command Line Reference | Song Player — Spring Boot

Everything you need to manage your Kubernetes cluster and Helm releases from the command line — flags explained, commands grouped by task, with your Song Player project in mind.

Chapter 1

Understanding Flags

-n -w -f -o -l -c --previous

Flags

Flags modify how a command behaves. They always start with **-** (short) or **--** (long). You can combine multiple flags in one command.

-n <namespace> — **Target a specific namespace**

Without -n, kubectl looks in the 'default' namespace. Your chart is in 'song-app' so always add -n song-app.

```
kubectl get pods -n song-app
kubectl get services -n song-app
kubectl logs <pod-name> -n song-app
```

-w — **Watch — live auto-refresh table**

Keeps the terminal open and updates the table whenever a pod/deployment status changes. Stop with Ctrl+C.

```
kubectl get pods -n song-app -w
kubectl get deployments -n song-app -w
```

Output updates automatically:

```
NAME                     READY   STATUS              RESTARTS
api-gateway-7d6b-xk2p9   0/1     ContainerCreating   0
api-gateway-7d6b-xk2p9   0/1     Running             0
api-gateway-7d6b-xk2p9   1/1     Running             0   <-- Ready!
```

-f — **Follow — stream logs live**

Like 'tail -f' on Linux. Keeps streaming new log lines as your app produces them. Stop with Ctrl+C.

```
kubectl logs <pod-name> -n song-app -f
kubectl logs -l app=api-gateway -n song-app -f
```

-o wide — **Output — extra columns (IP, node, etc.)**

Shows more detail like node name, pod IP. Use -o yaml for full YAML, -o json for JSON.

```
kubectl get pods -n song-app -o wide
kubectl get deployments -n song-app -o wide
kubectl get pod <pod-name> -n song-app -o yaml
```

-l <label> — **Label selector — filter by label**

Select pods/resources by their label instead of exact pod name. Useful since pod names have random suffixes.

```
kubectl get pods -l app=api-gateway -n song-app
kubectl logs -l app=eureka -n song-app -f
kubectl delete pod -l app=kafka -n song-app
```

-c <container> — **Container — target one container in a multi-container pod**

When a pod runs more than one container, use -c to pick which one you want logs or shell from.

```
kubectl logs <pod-name> -c api-gateway -n song-app
kubectl logs <pod-name> -c api-gateway -n song-app -f
kubectl exec -it <pod-name> -c api-gateway -n song-app -- bash
```

--previous — **Logs of a crashed/previous container**

If your pod crashed and restarted, --previous shows the logs from BEFORE the crash — very useful for debugging.

```
kubectl logs <pod-name> -n song-app --previous
kubectl logs <pod-name> -c api-gateway -n song-app --previous
```

■ TIP Combine flags freely: kubectl logs -l app=api-gateway -n song-app -f --tail=100

Chapter 2

kubectl Commands

Pods · Deployments · Services · Logs · SSH

Context & Cluster Setup

```
kubectl config get-contexts                    # see all clusters
kubectl config use-context <cluster-name>      # switch cluster
kubectl config current-context                 # which cluster am I on?
kubectl config set-context --current --namespace=song-app  # set default ns
```

Namespaces

```
kubectl get namespaces                         # list all namespaces
kubectl create namespace song-app              # create your namespace
```

Pods

```
kubectl get pods -n song-app                   # list all pods
kubectl get pods -n song-app -o wide           # with IP and node info
kubectl get pods -n song-app -w                # live watch
kubectl get pods -l app=api-gateway -n song-app  # filter by label
kubectl describe pod <pod-name> -n song-app    # full details of a pod
kubectl delete pod <pod-name> -n song-app      # delete a pod (auto-restarts)
```

Pod with Multiple Containers

Your pods may run more than one container. Here is how to inspect them:

```
# Step 1: See all containers inside a pod
kubectl describe pod <pod-name> -n song-app
# Look for the 'Containers:' section in the output

# Step 2: Get logs from a specific container
kubectl logs <pod-name> -c <container-name> -n song-app
kubectl logs <pod-name> -c <container-name> -n song-app -f

# Step 3: SSH into a specific container
kubectl exec -it <pod-name> -c <container-name> -n song-app -- bash
```

Deployments

```
kubectl get deployments -n song-app            # list all deployments
kubectl get deployments -n song-app -o wide    # with extra info
kubectl get deployments -n song-app -w         # live watch
kubectl describe deployment api-gateway -n song-app

# Restart a deployment (re-pulls image, recreates pods)
kubectl rollout restart deployment/api-gateway -n song-app
kubectl rollout restart deployment/eureka -n song-app
kubectl rollout restart deployment/kafka -n song-app

# Watch rollout progress
kubectl rollout status deployment/api-gateway -n song-app

# Rollback if something breaks
kubectl rollout undo deployment/api-gateway -n song-app
kubectl rollout history deployment/api-gateway -n song-app
```

Deployment Status Columns Explained:

Column	Meaning	Good Value
READY	running pods / desired pods	3/3 (all match)
UP-TO-DATE	pods updated to latest version	Same as READY count
AVAILABLE	pods accepting traffic	Same as READY count
AGE	how long deployment has been running	Any value

Services

```
kubectl get services -n song-app               # list all services
kubectl get svc -n song-app                    # 'svc' is short for services
kubectl describe svc api-gateway-service -n song-app
```

Logs

```
# By pod name
kubectl logs <pod-name> -n song-app
kubectl logs <pod-name> -n song-app -f             # live follow
kubectl logs <pod-name> -n song-app --tail=100     # last 100 lines
kubectl logs <pod-name> -n song-app --previous     # crashed pod logs

# By label (no need to know exact pod name)
kubectl logs -l app=api-gateway -n song-app -f
kubectl logs -l app=eureka -n song-app -f
kubectl logs -l app=kafka -n song-app -f
kubectl logs -l app=elasticsearch -n song-app -f
kubectl logs -l app=search -n song-app -f
kubectl logs -l app=security -n song-app -f
kubectl logs -l app=yt-ai -n song-app -f
kubectl logs -l app=yt-s3 -n song-app -f
```

SSH Into a Container

```
# Step 1: Get the pod name
kubectl get pods -n song-app

# Step 2: SSH in
kubectl exec -it <pod-name> -n song-app -- bash

# If bash not available (alpine-based image)
kubectl exec -it <pod-name> -n song-app -- sh

# SSH into specific container in a multi-container pod
kubectl exec -it <pod-name> -c <container-name> -n song-app -- bash

# Run a single command without interactive shell
kubectl exec <pod-name> -n song-app -- env
kubectl exec <pod-name> -n song-app -- cat /etc/hosts
```

Port Forward — Access Services Locally

```
# Access API Gateway locally
kubectl port-forward svc/api-gateway-service 8080:8080 -n song-app

# Access Eureka Dashboard
kubectl port-forward svc/eureka-service 8761:8761 -n song-app

# Access Elasticsearch
kubectl port-forward svc/elasticsearch-headless 9200:9200 -n song-app

# Access Kafka
kubectl port-forward svc/kafka-service 9092:9092 -n song-app

# Access any pod directly (not via service)
kubectl port-forward pod/<pod-name> 8080:8080 -n song-app
```

■ TIP After port-forward, open http://localhost: in your browser. Stop with Ctrl+C.

Debugging & Events

```
# See recent cluster events (great for debugging)
kubectl get events -n song-app --sort-by=.lastTimestamp

# See everything in namespace at once
kubectl get all -n song-app
```

Pod Status	Meaning	What To Do
Running	All good	Nothing needed
CrashLoopBackOff	App keeps crashing	kubectl logs --previous -n song-app
Pending	Cannot be scheduled	kubectl describe pod -n song-app

ImagePullBackOff	Wrong image name / no registry access	Check image name in values.yaml
OOMKilled	Out of memory — hit memory limit	Increase memory limit in values.yaml
Terminating	Pod is being deleted	Wait or force: kubectl delete pod --force

Chapter 3

Helm Commands

Install · Upgrade · Rollback · Debug

Install Your Chart

```
# Go into your chart directory first
cd ~/IdeaProjects/Song-Player---Spring-Boot/application-chart

# Dry run first — preview without deploying
helm install song-player . --dry-run --debug -n song-app --create-namespace

# Actual install
helm install song-player . -n song-app --create-namespace

# Install with custom values file
helm install song-player . -f values.yaml -n song-app --create-namespace
```

Upgrade After Changes

```
# Upgrade
helm upgrade song-player . -n song-app

# Safe upgrade — auto rollback if anything fails
helm upgrade song-player . -n song-app --atomic --timeout 5m

# Install if not exists, upgrade if already installed (most used)
helm upgrade --install song-player . -n song-app --create-namespace

# Upgrade with custom values
helm upgrade song-player . -f values.yaml -n song-app --atomic
```

Check Releases

```
helm list -n song-app                          # list releases in namespace
helm list -A                                   # all releases all namespaces
helm status song-player -n song-app            # current status
helm history song-player -n song-app           # all revisions
helm get values song-player -n song-app        # values used
helm get manifest song-player -n song-app      # actual K8s YAML deployed
```

Rollback

```
# See revision history first
helm history song-player -n song-app

# Rollback to previous revision
helm rollback song-player -n song-app

# Rollback to a specific revision number
helm rollback song-player 2 -n song-app
```

Lint & Debug

```
# Check chart for errors
helm lint .

# Preview generated YAML without deploying
helm template . -f values.yaml

# Preview with release name
helm template song-player . -f values.yaml -n song-app
```

Uninstall

```
helm uninstall song-player -n song-app
```

■■ NOTE

Uninstall removes all K8s resources created by the chart but does NOT delete PersistentVolumeClaims. Delete those manually if needed.

Chapter 4

Quick Reference

Cheatsheet for your Song Player project

Your Daily Workflow

```
# 1. Go to chart directory
cd ~/IdeaProjects/Song-Player---Spring-Boot/application-chart

# 2. Deploy / update
helm upgrade --install song-player . -n song-app --create-namespace

# 3. Watch pods come up
kubectl get pods -n song-app -w

# 4. Follow logs of a service
kubectl logs -l app=api-gateway -n song-app -f

# 5. Access locally
kubectl port-forward svc/api-gateway-service 8080:8080 -n song-app
```

All Flags at a Glance

Flag	Full Name	Used With	What It Does
-n song-app	namespace	almost every command	target song-app namespace
-w	watch	kubectl get	live auto-refresh table
-f	follow	kubectl logs	stream logs live
-o wide	output	kubectl get	show extra columns
-l app=X	label	get / logs / delete	filter by label
-c name	container	logs / exec	target one container
--previous	previous	kubectl logs	logs of crashed pod
--tail=100	tail	kubectl logs	last N lines only
--dry-run	dry-run	helm install/upgrade	preview without deploying
--atomic	atomic	helm upgrade	auto-rollback on failure

Services in Your Chart

Service	Port Forward Command	Browser URL

api-gateway	kubectl port-forward svc/api-gateway-service 8080:8080 -n song-app	localhost:8080
eureka	kubectl port-forward svc/eureka-service 8761:8761 -n song-app	localhost:8761
elasticsearch	kubectl port-forward svc/elasticsearch-headless 9200:9200 -n song-app	localhost:9200
kafka	kubectl port-forward svc/kafka-service 9092:9092 -n song-app	localhost:9092

Happy Deploying! ■